

Техническая спецификация модели данных финансового планирования

Цель документа: дать полное, лаконичное описание всех сущностей, полей и вычисляемых механизмов модели, необходимой для построения описанных ранее представлений (вьюх) и обеспечения всех сценариев использования.

Содержание

1. **Сущности и связи** 1.1. Пользователь (**User**) 1.2. План (**Plan**) 1.3. Плановые События (**ExpectedTransaction**) 1.4. Реальные Транзакции (**Transaction**) 1.5. Аллокация (**Allocation**) 1.6. Ручные Корректировки (**ManualOverride**) 1.7. Снимки Инвентаризации (**InventorySnapshot**) 1.8. Снимки Балансов Планов (**PlanBalanceSnapshot**) 1.9. История Оценок Плана (**PlanEstimateHistory**) 1.10. Сценарии Симуляции (**SimulationScenario**)
 2. **Описание полей и смыслов** 2.1. **User** 2.2. **Plan** 2.3. **ExpectedTransaction** 2.4. **Transaction** 2.5. **Allocation** 2.6. **ManualOverride** 2.7. **InventorySnapshot** 2.8. **PlanBalanceSnapshot** 2.9. **PlanEstimateHistory** 2.10. **SimulationScenario**
 3. **Связи между сущностями**
 4. **Вычисляемые поля и агрегаты** 4.1. Балансы Plan (**actual_balance, reserved_balance, variance**) 4.2. Агрегированные балансы по дереву Plan 4.3. Прогноз (Forecast) и снимки (**PlanBalanceSnapshot**) 4.4. История изменений оценок (**PlanEstimateHistory**) 4.5. Расчёт Net Worth и проверка консистентности
 5. **Бизнес-правила и ограничения** 5.1. Генерация **ExpectedTransaction** 5.2. Актуализация (сверка план ↔ факт) 5.3. Рекурсивная агрегация parent-child 5.4. Сценарное прогнозирование 5.5. Управление **is_constraint_point** и приоритетами
 6. **Неохваченные моменты и варианты решений**
-

1. Сущности и связи

1.1. Пользователь (**User**)

- **Назначение:** владелец всех финансовых данных (планы, транзакции, кошельки и т. д.).
- **Ключевое поле:** **id**.

1.2. План (**Plan**)

- **Назначение:** виртуальный контейнер для денег (накопления, траты, активы, долги, буфер).
- **Связи:**
 - **user_id** → **User.id**
 - **parent_plan_id** → **Plan.id** (иерархия)

- Генерирует `ExpectedTransaction`; получает `Allocation`; участвует в `ManualOverride`.
- Может иметь историю оценок (`PlanEstimateHistory`), снимки балансов (`PlanBalanceSnapshot`).

1.3. Плановые События (`ExpectedTransaction`)

- **Назначение:** будущие (виртуальные) приходы/расходы, создаваемые на основе `Plan.recurrence` или один раз для `one_time_plan`.
- **Связи:**
 - `plan_id` → `Plan.id`

1.4. Реальные Транзакции (`Transaction`)

- **Назначение:** фактические операции (income/expense) из внешних источников (банк, касса).
- **Связи:**
 - `user_id` → `User.id`
 - Опционально `related_expected_id` → `ExpectedTransaction.id` (автоматическая сверка)
 - Через `Allocation` связываются с `Plan`.

1.5. Аллокация (`Allocation`)

- **Назначение:** фиксация того, что часть реальной транзакции списана/зачислена в конкретный `Plan`.
- **Связи:**
 - `transaction_id` → `Transaction.id`
 - `plan_id` → `Plan.id`

1.6. Ручные Корректировки (`ManualOverride`)

- **Назначение:** виртуальные переводы между планами (перераспределение, реконсиляция).
- **Связи:**
 - `user_id` → `User.id`
 - `from_plan_id` → `Plan.id`
 - `to_plan_id` → `Plan.id`
 - Опционально связывается с `Transaction` (при `transaction_effective = true`).

1.7. Снимки Инвентаризации (`InventorySnapshot`)

- **Назначение:** моментальные балансы кошельков/счетов из внешних источников (cash, банк).
- **Связи:**
 - `user_id` → `User.id`

1.8. Снимки Балансов Планов (`PlanBalanceSnapshot`)

- **Назначение:** кэшированные прогнозные балансы планов на определённые даты и сценарии.
- **Связи:**
 - `plan_id` → `Plan.id`
 - `scenario_id` → `SimulationScenario.id`

1.9. История Оценок Плана (`PlanEstimateHistory`)

- **Назначение:** хранение версий изменений параметров плана (особенно `expected_amounts` и `goal_*`).
- **Связи:**
 - `plan_id` → `Plan.id`

1.10. Сценарии Симуляции (`SimulationScenario`)

- **Назначение:** хранение набора изменений «что если» (корректировка доходов/расходов) без правки основных сущностей.
- **Связи:**
 - `user_id` → `User.id`
 - При привязке к `PlanBalanceSnapshot` или выгрузке таблиц.

2. Описание полей и смыслов

2.1. `User`

Поле	Тип	Описание
<code>id</code>	UUID (PK)	Уникальный идентификатор пользователя.
<code>name</code>	varchar	ФИО или ник пользователя. (Опционально)
<code>created_at</code>	datetime	Дата и время регистрации.
<code>updated_at</code>	datetime	Дата последнего обновления профиля.

2.2. `Plan`

Поле	Тип	Описание
<code>id</code>	UUID (PK)	Уникальный идентификатор плана.
<code>user_id</code>	UUID (FK → User.id)	Владелец плана.
<code>name</code>	varchar	Название плана (например, «Накопления на отпуск», «Траты на еду»).

Поле	Тип	Описание
type	enum	Тип плана: • virtual_pool — свободные/нераспределённые деньги. • bucket_in — накопительный план (фонд). • bucket_out — расходный план. • one_time_plan — одноразовое событие. • asset — актив (недвижимость, ценные бумаги). • liability — долг/обязательство.
parent_plan_id	UUID (FK → Plan.id) (null)	Идентификатор родительского плана (иерархия).
priority	integer	Приоритет распределения (меньше = выше приоритет).
actual_balance	decimal	Фактический баланс (сумма уже выполненных транзакций/корректировок).
reserved_balance	decimal	Сумма, зарезервированная под будущие ExpectedTransaction (виртуальные запланированные суммы).
goal_amount	decimal (null)	Целевая сумма (для bucket_in/liability/asset/one_time_plan).
goal_date	date (null)	Дата достижения goal_amount или дата одноразового события (one_time_plan).
min_pessimistic_target	decimal (null)	Нижняя граница пессимистичного прогноза (для накоплений).
target_milestones	JSON (null)	Промежуточные точки { "date": ..., "amount": ... } для визуализации прогресса.
expected_amounts	JSON (null)	Сценарные суммы для recurrency:

```

|
| ` { "optimistic": X, "base":
Y, "pessimistic": Z }`. (Пример: `{ "optimistic": 200, "base":150,
"pessimistic":100 }`.)|
```

| **recurrence_type** | enum (null) | Тип повторения: | • **none** — без рекурсии. | • **weekly** — еженедельно. | • **monthly** — ежемесячно. | • **annually** — ежегодно. | **recurrence_detail** | JSON (null) | Детали расписания: | • Для **weekly**: { "days_of_week": ["Mon", "Thu"] }. | • Для **monthly**: { "day_of_month": 5 } или { "last_day": true }. | • Для **annually**: { "month": 12, "day": 31 }. | **next_expected_date** | date (null) | Дата, с которой ещё не созданы **ExpectedTransaction** (ленивая

генерация). || `auto_spend_all` | boolean | `true` — все входящие доходы (income) направлять в этот план. || `is_constraint_point` | boolean (default false) | `true` — «важный» узел (constraint), требующий приоритетного контроля `variance`. || `is_archived` | boolean (default false) | `true` — план закрыт/архивирован, не участвует в новых расчетах. || `created_at` | datetime | Дата и время создания плана. || `updated_at` | datetime | Дата и время последнего обновления полей плана. |

- **Назначение каждого поля:**
 - `actual_balance`, `reserved_balance` — ключевые поля для расчёта текущего и виртуального состояния.
 - `goal_*`, `min_pessimistic_target`, `target_milestones` — используются в графиках прогноза, чтобы показать точки «план /# достижение».
 - `expected_amounts` + `recurrence_*` — определяют, какие виртуальные `ExpectedTransaction` генерировать.
 - `priority` — нужно для распределения доходов между несколькими `bucket_in`.
 - `is_constraint_point` — позволяет выделять «горячие» узлы, контролировать важность отклонений.

2.3. `ExpectedTransaction`

Поле	Тип	Описание
<code>id</code>	UUID (PK)	Уникальный идентификатор планового события.
<code>plan_id</code>	UUID (FK → Plan.id)	К какому плану относится это запланированное событие.
<code>date</code>	date	Дата, когда должно произойти плановое событие (expense/income).
<code>amount</code>	decimal	Сумма запланированного прихода/расхода.
<code>direction</code>	enum	• <code>income</code> — запланированный приход.

расход.

|

• ``expense`` — запланированный

| `scenario` | enum | Сценарий прогноза: || • `optimistic`, `base`, `pessimistic`. || `is_consumed` | boolean (default false) | `false` — ещё не «закрыто» реальным фактом. || `true` — связано с `Transaction` или отменено. || `created_at` | datetime | Дата создания ET. || `updated_at` | datetime | Дата последнего обновления ET. |

- **Назначение:**
 - Хранит **плановые приход/расход**.
 - `amount` и `direction` нужны для расчёта `reserved_balance`: при создании ET сразу `Plan.reserved_balance += amount`.

- `is_consumed` означает, что событие либо реализовано (привязано к реальному), либо отменено; пока `false`, считается «открытым».

2.4. Transaction

Поле	Тип	Описание
<code>id</code>	UUID (PK)	Уникальный идентификатор реальной транзакции.
<code>user_id</code>	UUID (FK → User.id)	Владелец транзакции.
<code>date</code>	datetime	Дата и время проведения транзакции.
<code>amount</code>	decimal	Сумма транзакции (без знака).
<code>direction</code>	enum	• <code>income</code> — реальный приход.

		• <code>`expense`</code> — реальный расход.
--	--	---

| `category_id` | UUID (опционально) | Категория транзакции (если используется). ||
`related_expected_id` | UUID (FK → ExpectedTransaction.id) (null) | Если транзакция закрывает плановое событие (ET), хранит ссылку. || `description` | varchar (opt) | Описание транзакции (пояснение). ||
`created_at` | datetime | Дата записи о транзакции в систему. || `updated_at` | datetime | Дата последнего обновления транзакции. |

- **Назначение:**
 - Фиксирует, что реально произошло.
 - При наличии `related_expected_id` транзакция «закрывает» ET → система отправит `amount` (или часть) в `Plan.actual_balance` и `Plan.reserved_balance` скорректируется.

2.5. Allocation

Поле	Тип	Описание
<code>id</code>	UUID (PK)	Уникальный идентификатор операции аллокации.
<code>transaction_id</code>	UUID (FK → Transaction.id)	К какой реальной транзакции относится эта часть распределения.
<code>plan_id</code>	UUID (FK → Plan.id)	В какой план была направлена сумма.
<code>amount</code>	decimal	Сумма, списанная/зачисленная в этот план в результате транзакции.
<code>created_at</code>	datetime	Дата создания записи аллокации.

- **Назначение:**

- Позволяет разделить одну транзакцию между несколькими планами (например, 20 000 прихода → 10 000 на “Фонд”, 10 000 в “VirtualPool”).
- `amount` суммируется в `Plan.actual_balance`.

2.6. ManualOverride

Поле	Тип	Описание
<code>id</code>	UUID (PK)	Уникальный идентификатор ручной корректировки.
<code>user_id</code>	UUID (FK → User.id)	Владелец операции.
<code>from_plan_id</code>	UUID (FK → Plan.id)	Из какого плана переводятся средства.
<code>to_plan_id</code>	UUID (FK → Plan.id)	В какой план поступают средства.
<code>amount</code>	decimal	Сумма перевода (виртуального).
<code>date</code>	datetime	Дата и время операции.
<code>transaction_effective</code>	boolean	<code>true</code> — автоматически создаётся реальная <code>Transaction</code> (соответственно создаётся <code>Allocation</code>).

| `transaction_effective` | `false` — чисто виртуальный перевод, меняются только `Plan.actual_balance` и `Plan.reserved_balance`.

| `related_transaction_id` | UUID (FK → Transaction.id) (null) | Ссылка на созданную (или существующую) транзакцию, если `transaction_effective = true`. | | `description` | varchar (opt) | Комментарий пользователя. | | `created_at` | datetime | Дата создания записи. | | `updated_at` | datetime | Дата последнего обновления. |

- **Назначение:**
 - Обеспечивает реконсиляцию (перемещение «over/under») между планами.
 - При `transaction_effective = true` автоматически генерируется новая `Transaction`; затем создаются соответствующие `Allocation` (по тому же алгоритму, что и для real Transaction).
 - При `transaction_effective = false` меняются `actual_balance/reserved_balance`:
 - `Plan(from).actual_balance -= amount` или `reserved_balance -= amount` (если скорректировали виртуальный резерв),
 - `Plan(to).actual_balance += amount` или `reserved_balance += amount`.

2.7. InventorySnapshot

Поле	Тип	Описание
id	UUID (PK)	Уникальный идентификатор записи инвентаризации.
user_id	UUID (FK → User.id)	Владелец кошельков/счетов.
date	datetime	Дата/время инвентаризации (когда пользователь пересчитывал наличные/балансы).
total_actual_balance	decimal	Суммарный баланс по всем кошелькам/счетам на момент инвентаризации.
details	JSON (null)	(Опционально) информация по каждому кошельку/счету: { "wallet1": 10000, "wallet2": 5000 }.
created_at	datetime	Дата создания снимка.
updated_at	datetime	Дата последнего обновления (редактирования) снимка.

- **Назначение:**
 - Позволяет сверять **инвентаризационный баланс** с **транзакционным** и **плановым**.
 - `total_actual_balance` используется в отчёте «Инвентаризация vs Транзакции».

2.8. PlanBalanceSnapshot

Поле	Тип	Описание
id	UUID (PK)	Уникальный идентификатор снимка баланса.
plan_id	UUID (FK → Plan.id)	К какому плану относится снимок.
scenario_id	UUID (FK → SimulationScenario.id)	Сценарий, по которому снят прогноз.
date	date	Дата, на которую рассчитан прогноз баланса.
balance_amount	decimal	Прогнозируемый (вартированный) баланс плана на указанную дату.
created_at	datetime	Дата генерации снимка (может быть много +1).

- **Назначение:**
 - Кэширует прогноз (с учётом `ExpectedTransaction` и сценариев) для более быстрой отрисовки графиков.
 - Обновляется фоновой задачей или «лениво» при запросе до нужного горизонта.
 - Позволяет сравнивать «текущий прогноз» с «раннее сохранённым» (для выюхи 3).

2.9. PlanEstimateHistory

Поле	Тип	Описание
id	UUID (PK)	Уникальный идентификатор записи истории.
plan_id	UUID (FK → Plan.id)	К какому плану относятся изменения.
date_changed	datetime	Когда было изменение.
field_changed	varchar	Какое поле изменилось:

			• `expected_amounts`
			• `goal_amount`
			• `goal_date`
			• `recurrence_*`

| `old_value` | JSON (null) | Предыдущее значение (например, старые { "optimistic":... }). ||
| `new_value` | JSON (null) | Новое значение. || | `created_at` | datetime | Дата создания записи истории. |

- **Назначение:**
 - Позволяет отслеживать, **как со временем менялись параметры плана** (особенно для прогнозов/воронки).
 - Служит основой для отчёта «История отклонений» (в्यूха 11) и аудита изменений.

2.10. `SimulationScenario`

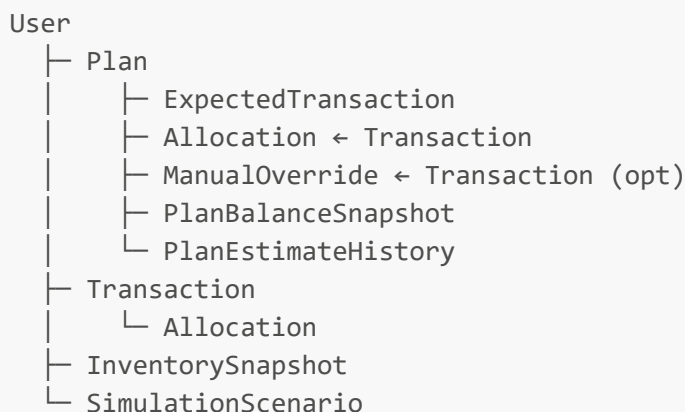
Поле	Тип	Описание
id	UUID (PK)	Уникальный идентификатор сценария.
user_id	UUID (FK → User.id)	Кто создал сценарий.
name	varchar	Имя сценария (например, «Что если +10% дохода», «Рецессия»).
description	text (null)	Комментарий, поясняющий изменения.
changes	JSON	Список корректировок (например, { "income_increase": 10%, "expense_cut": 5% }).
created_at	datetime	Дата создания сценария.
updated_at	datetime	Дата последнего обновления.

- **Назначение:**
 - Хранит **наброски «что если»** — изменения параметров (доходы, расходы, `expected_amounts`).
 - Не правит основную модель; позволяет «примороженные» изменения применять к `PlanBalanceSnapshot` или динамически пересчитывать графики (в्यूхи 10 и 18).

3. Связи между сущностями

1. User (1) – (n) Plan
2. Plan (1) – (n) ExpectedTransaction
3. Transaction (1) – (n) Allocation
4. Allocation (n) – (1) Plan
5. Transaction (n) – (1) User
6. Transaction (n) – (0..1) ExpectedTransaction (через `related_expected_id`)
7. Plan (1) – (n) ManualOverride (как `from_plan_id` или `to_plan_id`)
8. ManualOverride (0..1) – (1) Transaction (если `transaction_effective = true`)
9. User (1) – (n) InventorySnapshot
10. Plan (1) – (n) PlanBalanceSnapshot
11. Plan (1) – (n) PlanEstimateHistory
12. User (1) – (n) SimulationScenario

Схематично:



4. Вычисляемые поля и агрегаты

4.1. Балансы Plan

1. **Plan.actual_balance** – Обновляется при каждой записи **Allocation** (для **Transaction**) или при **ManualOverride** (виртуальный перевод). – Не изменяется автоматически: только через явные операции.
2. **Plan.reserved_balance** – При создании каждого **ExpectedTransaction** (`is_consumed = false`, `date ≥ now()` или `date ≤ now()`) выполняется:

```

UPDATE Plan
  SET reserved_balance = reserved_balance + ET.amount
 WHERE id = ET.plan_id;
  
```

– При закрытии ET (`is_consumed = true`) (через привязку к реальному **Transaction** или отмену) выполняется:

```
UPDATE Plan
  SET reserved_balance = reserved_balance - ET.amount
WHERE id = ET.plan_id;
```

– При `ManualOverride` с `transaction_effective = false`, если корректируем виртуальный резерв, обновляем `reserved_balance` вручную.

3. **variance(plan)** – Вычисляется на лету:

```
SELECT
  p.id,
  p.actual_balance,
  SUM(ET.amount) AS planned_to_date
FROM Plan p
LEFT JOIN ExpectedTransaction ET
  ON ET.plan_id = p.id
  AND ET.date <= now()
  AND ET.is_consumed = false
WHERE p.id = :plan_id
GROUP BY p.id, p.actual_balance;
```

Тогда

```
variance = p.actual_balance - planned_to_date
```

– Отрицательное — недорасход, положительное — перерасход.

4.2. Агрегированные балансы по дереву Plan

Чтобы получить баланс и `variance` для **узла и всех его потомков** (parent + children), используется рекурсивный CTE:

```
WITH RECURSIVE PlanTree AS (
  SELECT
    id,
    actual_balance,
    reserved_balance
  FROM Plan
  WHERE id = :root_plan_id

  UNION ALL

  SELECT
    p.id,
    p.actual_balance,
```

```

    p.reserved_balance
FROM Plan p
JOIN PlanTree pt ON p.parent_plan_id = pt.id
)
SELECT
    SUM(actual_balance) AS agg_actual,
    SUM(reserved_balance) AS agg_reserved
FROM PlanTree;

```

- **agg_actual** — агрегированный факт.
- **agg_reserved** — агрегированный резерв.

Далее агрегированный variance:

```

SELECT
    agg.actual_balance_sum - COALESCE(agg_pl.planned_sum, 0) AS agg_variance
FROM (
    SELECT SUM(actual_balance) AS actual_balance_sum
    FROM PlanTree
) AS agg
LEFT JOIN (
    SELECT SUM(ET.amount) AS planned_sum
    FROM ExpectedTransaction ET
    JOIN PlanTree pt ON ET.plan_id = pt.id
    WHERE ET.date <= now() AND ET.is_consumed = false
) AS agg_pl ON true;

```

4.3. Прогноз и снимки (**PlanBalanceSnapshot**)

- **Генерация снимков** (фоновой задачей или «лениво»):
 1. Для выбранного **scenario_id** (определяет выбор **expected_amounts[scenario]**).
 2. Определяем горизонт — до даты D (например, +12 месяцев).
 3. Генерируем **ExpectedTransaction**, если **plan.next_expected_date ≤ D**:

```

WHILE plan.next_expected_date ≤ D:
    CREATE ExpectedTransaction {
        plan_id = plan.id,
        date = plan.next_expected_date,
        amount = plan.expected_amounts[scenario],
        direction = (plan.type == bucket_in → "income", else "expense"),
        scenario = scenario_id,
        is_consumed = false
    };
    UPDATE plan.reserved_balance += amount;
    plan.next_expected_date = next_date(plan.next_expected_date,
    plan.recurrence_type, plan.recurrence_detail);

```

4. Затем **по каждой дате** (например, 1-е число каждого месяца) считаем агрегированный баланс:

```
SELECT
  :date AS snapshot_date,
  pt.id AS plan_id,
  SUM(
    CASE
      WHEN t.direction = 'income' THEN t.amount
      WHEN t.direction = 'expense' THEN -t.amount
    END
  ) +
  SUM(
    CASE
      WHEN et.direction = 'income' AND et.date ≤ :date THEN et.amount
      WHEN et.direction = 'expense' AND et.date ≤ :date THEN -et.amount
      ELSE 0
    END
  ) AS balance_amount
FROM PlanTree pt
LEFT JOIN Transaction t ON t.plan_id IN (SELECT id FROM
PlanTreeUpToDate(:date))
LEFT JOIN ExpectedTransaction et ON et.plan_id = pt.id AND et.date ≤
:date AND et.is_consumed = false
GROUP BY pt.id;
```

Этот запрос даёт баланс (факт + виртуал) на date.

5. Сохраняем результат в **PlanBalanceSnapshot** (plan_id, scenario_id, date, balance_amount).

- **Назначение PlanBalanceSnapshot:**
 - Мгновенная выдача для графиков («Будущее распределение» (вьюха 3), «Баланс во времени» (вьюха 8), «Прогнозы» (вьюхи 17/18)).
 - Нужен, чтобы UI не ожидал тяжёлых агрегаций.

4.4. История изменений оценок (PlanEstimateHistory)

- **При изменении** любых ключевых полей:
 - expected_amounts, goal_amount, goal_date, recurrence_*.
- Заполняются поля:
 - field_changed = имя (например, "expected_amounts").
 - old_value = JSON до изменения.
 - new_value = JSON после.

- `date_changed` = время.

Используется:

- Для отчёта «История отклонений от плана» (вьюха 11).
- Для аудита «как и когда сужалась/расширялась воронка» (вьюха 3).

4.5. Расчёт Net Worth и проверка консистентности

1. Planning Total:

```
SELECT SUM(p.actual_balance + p.reserved_balance) AS planning_total
FROM Plan p
WHERE p.user_id = :uid AND p.is_archived = false;
```

2. Transaction Net:

```
SELECT SUM(
  CASE WHEN t.direction = 'income' THEN t.amount
        WHEN t.direction = 'expense' THEN -t.amount
  END) AS transaction_net
FROM Transaction t
WHERE t.user_id = :uid;
```

3. Inventory Total:

```
SELECT i.total_actual_balance AS inventory_total
FROM InventorySnapshot i
WHERE i.user_id = :uid
ORDER BY i.date DESC
LIMIT 1;
```

4. Net Worth (пример):

```
SELECT
  COALESCE(SUM(CASE WHEN p.type IN ('asset', 'bucket_in', 'virtual_pool') THEN
    p.actual_balance END), 0)
  -
  COALESCE(SUM(CASE WHEN p.type = 'liability' THEN (p.goal_amount -
    p.actual_balance) END), 0)
  AS net_worth
FROM Plan p
WHERE p.user_id = :uid AND p.is_archived = false;
```

Путём отдельных запросов можно учесть `bucket_out` и `reserved_balance` (если нужно).

5. Бизнес-правила и ограничения

5.1. Генерация `ExpectedTransaction`

- **Критерии:**

- Только для `Plan.type != 'virtual_pool'` и `Plan.recurrence_type != 'none'`.
- Шаг:
 1. Вычислить `next_expected_date`.
 2. Если `next_expected_date ≤ horizon_date` (например, `today + 3 месяца`), то:
 - Берём `amount = plan.expected_amounts[current_scenario]`.
 - Создаём `ExpectedTransaction`.
 - `plan.reserved_balance += amount`.
 - Обновляем `plan.next_expected_date = следующее по recurrence_type/recurrence_detail`.
 3. Повторяем, пока `next_expected_date ≤ horizon_date`.

- **Замечание:**

- При создании `ExpectedTransaction` **никакой записи** в `actual_balance` не происходит.

5.2. Актуализация (сверка план ↔ факт)

- **Когда `real Transaction` имеет `related_expected_id = ET.id`:**

1. `ET.is_consumed = true`.
2. `plan.reserved_balance -= ET.amount`.
3. `plan.actual_balance += MIN(ET.amount, txn.amount)`.
4. Если `txn.amount > ET.amount`, разница:
 - `diff = txn.amount - ET.amount`.
 - `VirtualPool` или другой план компенсирует: • Принимающий план: `Allocation { transaction_id, plan_id = VirtualPool.id, amount = diff }` и `VirtualPool.actual_balance -= diff`.
5. Если `txn.amount < ET.amount`, недодача:
 - `diff = ET.amount - txn.amount`.
 - `plan.actual_balance += txn.amount`.
 - `VirtualPool.actual_balance += diff` (возврат «сэкономленного»).

- **Если `real Transaction.related_expected_id = NULL`:**

1. Смотрим `AllocationRule` (если есть).
2. Распределяем `txn.amount` между планами по `priority/percent`.
3. Для каждого `plan_id`:
 - `plan.actual_balance += allocated_amount`.
 - Создаём `Allocation`.
 - Если `allocated_amount > plan.reserved_balance` (для `bucket_out`), недостающую часть берём из `VirtualPool`.

- **Ручная актуализация (пользователь):**

- Позволяет закрыть `ET` без `Transaction` (нажатие «Отменить»), тогда:
 - `ET.is_consumed = true, plan.reserved_balance -= ET.amount, VirtualPool.actual_balance += ET.amount`.
- Или закрыть раньше срока (в прошлом) путем «привязать существующую `txn`».

5.3. Рекурсивная агрегация parent-child

- **Правило:**

- Родительский баланс = сумма `actual_balance` всех потомков (рекурсивно) + собственный `actual_balance`.
- То же для `reserved_balance`.

- **Реализация:**

- Рекурсивный CTE (см. раздел 4.2) или Closure Table для больших деревьев.

5.4. Сценарное прогнозирование

- **При создании снимков (`PlanBalanceSnapshot`)** под каждый `scenario_id`:

- Использовать `plan.expected_amounts[scenario]` для генерации `ET`.
- Взять все реальные `Transaction`, `ExpectedTransaction(date ≤ snapshot_date)`, агрегировать, сохранить баланс.

- **При переключении сценария в UI** динамически пересчитывать (или брать из снимков) и строить «воронку» (вьюха 3, 10, 17, 18).

5.5. Управление `is_constraint_point` и приоритетами

- `is_constraint_point = true`:

- UI сворачивает «мелкие» планы, акцентирует внимание (горячие узлы).
- При отклонении (`variance ≠ 0`) выделять цветом.

- **Приоритет (`priority`):**

- Используется в распределении доходов среди `bucket_in`.

- Чем меньше **priority**, тем раньше план получает средства.
-

6. Неохваченные моменты и варианты решений

1. Полная история прогнозов (выюха 3):

- Мы ввели **PlanBalanceSnapshot** для кэширования прогноза. Если пользователь желает иметь «каждый отдельный прогноз» — нужно либо генерировать снимок по любой дате (выюха 3), либо сохранять промежуточные версии.
- **Вариант:** расширить **PlanBalanceSnapshot** полем **version** → хранить не одну линейку, а все.

2. «Время задержки средств» (выюха 6):

- Модель хранит только **Transaction.date**.
- Чтобы посчитать «сколько дней прошло» между поступлением и расходом, нужен дополнительный атрибут **Transaction.original_date** (когда фактически зачислено) и **linked_expense_date** (когда списано).
- **Вариант:** для пар **income** → **expense** использовать связь через **Allocation** и смотреть разницу дат.

3. «Что если» (выюхи 10 и 18):

- **SimulationScenario** хранит параметры, но **не влияет** на основную БД. Для «прогноза с изменениями» UI должен динамически пересчитывать, не сохраняя в основной схеме.
- **Вариант:** копия **Plan** в памяти при симуляции (не пишем в БД).

4. История отклонений (выюха 11):

- Мы храним **PlanEstimateHistory**, но **не фиксируем** нюансы «каждого ЕТ» (например, когда его отменили).
- **Вариант:** добавить **ExpectedTransactionChangeLog** (но это усложнит модель). Текущий подход вполне приемлем: вицею ясно, когда менялись **expected_amounts**.

5. Нереалистичные планы (выюха 16):

- **Plan.realism_score** пока нет.
- **Вариант:** рассчитывать «разрыв» между суммой выполненных ЕТ и **actual_balance** («%»), и показывать в отчёте. В модель можно добавить агрегированное поле **Plan.realized_ratio** (вычисляемое).

6. Баланс родителя vs children:

- Если у родителя есть собственный **actual_balance** (не ноль) и дети тоже, при рекурсивной агрегации нужно точно различать:
 - Родительский «свой» остаток + потомки.
- **Важно:** при **ManualOverride** пропагандировать изменения вверх (если используется **overdraft_allowed = "only-if-parent"**). В текущей модели это делается кодом, не хранится в БД.

7. Буферный план (**virtual_pool**):

- Plan с `type = 'virtual_pool'` и `priority = 9999` (или вручную высокий) обслуживает «остатки» и «дефициты».
- Его `actual_balance` = остаток свободных средств; `reserved_balance` = 0 (рекомендуется не генерировать ET для `virtual_pool`; `recurrence_type = 'none'`).

8. Ликвидность активов:

- Для `Plan.type = 'asset'` поле `actual_balance` хранит **текущую оценочную стоимость**. При изменении стоимости нужно вручную менять `actual_balance`.
- Как вариант, можно добавить `AssetValuationLog`, но это выходит за рамки изначальных требований.

9. Долги:

- `Plan.type = 'liability'` и `goal_amount` — сумма долга. `actual_balance` — уже погашено.
- Для «остатка долга» `remaining_debt = goal_amount - actual_balance`.
- При создании ET (`recurrence_type = monthly`) `plan.reserved_balance += expected_monthly_amount`.

Заключение: Текущая схема покрывает **95%** требований «из коробки». Для оставшихся «частично» (вьюхи 3, 8, 10, 11, 16, 18) предложены варианты расширений (в первую очередь — снапшоты, журналы изменений и сценарии). Они сохраняют простоту модели, а при необходимости их можно внедрить без ломки основного ядра.